Immutable Infrastructure

never fix a running server; replace it

Watch the 4-min video

In one line: instead of patching or reconfiguring running servers, build a new versioned, tested **image** with the change, start new servers from it, move traffic, and destroy the old ones; drift disappears, environments become identical, and rollback means redeploying the previous image.

1 The snowflake server

Server **web-07** has run for three years. It has been patched by hand two hundred times, someone installed a debugging tool in a hurry, and its config no longer matches the other servers. Nobody dares restart it, because nobody knows whether it would come back the same. That's a **snowflake**: unique, fragile, impossible to reproduce. Configuration management tools (Ansible, Chef, Puppet) reduce the problem by re-applying a desired config, but any change they don't manage still accumulates.

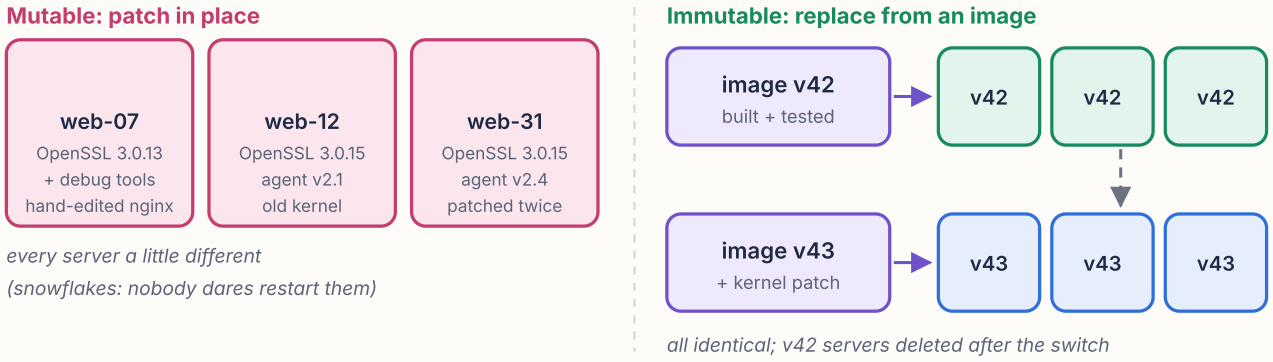


Figure 1. Mutable servers drift apart over time. Immutable servers are all built from one image version, and change only by being replaced.

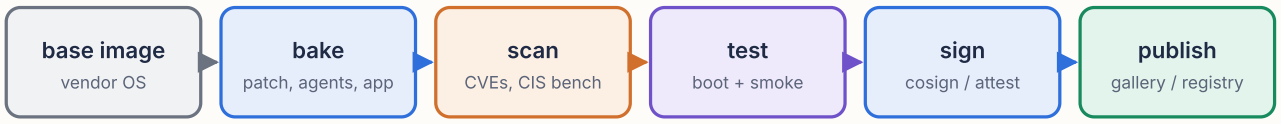
2 Never change a running server

The rule is simple: no logging in to patch, upgrade or reconfigure. Every change, from a kernel patch to a log-level setting baked into the image, goes through the same path: change code, build a new image, roll it out by replacement. Servers become **cattle, not pets**: numbered, identical, and replaceable without ceremony. Containers made this the default for applications (you never patch inside a running container); the same idea now applies to VMs and Kubernetes nodes.

	Mutable (patch in place)	Immutable (replace)
How changes happen	SSH, scripts, config management runs	new image version, rolling replacement
State of the fleet	drifts; each server has a history	identical per image version
Rollback	undo steps (often impossible)	redeploy the previous image
Security scanning	scan each server	scan the image once; it covers every server running it
Cost of a change	fast for one fix, risky at scale	build + rollout time (minutes to an hour)

3 Golden images

A **golden image** is a machine or container image built automatically by a pipeline, containing the OS, patches, security agents, configuration and often the application itself. **Packer** builds VM images (AMIs, Azure managed images) from a template; Azure VM Image Builder and EC2 Image Builder are managed equivalents; Dockerfiles or Buildpacks build container images. Each image gets a version, is published to a registry or image gallery, and is promoted through environments like any release.



Triggered by code changes AND on a schedule (new base image or CVE), so patching is a pipeline run, not a ticket.

Figure 2. An image pipeline. The same image version runs in dev, staging and production; only injected configuration and secrets differ.

Bake vs fry. "Baking" puts everything in the image, so boot is fast and nothing can fail at startup; "frying" configures at boot with cloud-init or a config tool, so images are fewer but boots are slower and less predictable. The usual balance: bake the OS, patches, agents and app; inject only **environment-specific** values (endpoints, feature flags, secrets from Key Vault) at start. Never bake secrets into images. Taking it further, immutable OSes like Bottlerocket, Flatcar and Talos have read-only root file systems and no package manager, and containers can set `readOnlyRootFilesystem: true`.

4 Deployment means replacement

A **rolling update** (VM scale set or auto-scaling group instance refresh, Kubernetes node pool upgrade), a **blue-green** switch, or a new node pool brings up servers from the new image; **health checks** confirm they work; traffic moves; the old servers are terminated. Rollback is deploying the previous image version, which you know worked, rather than trying to reverse a series of in-place steps.

5 Why it's worth it

- **No configuration drift**: server 1 and server 40 are identical; staging and production run the same bits.
- **Reproducible problems**: start the same image locally or in a test environment and you have the same server.
- **Security**: one image scan covers the fleet; attackers who change a server lose their foothold at the next replacement; you can say exactly which image every server runs, which simplifies audits.
- **Disaster recovery**: rebuilding an environment is starting images with IaC (lesson 61), not reconstructing years of hand changes.

6 New habits it requires

- **Nobody logs in to fix production.** SSH is off or break-glass only, audited; fixes go through the image pipeline. Debugging uses logs, metrics and traces, or an ephemeral debug container.
- **State and logs live outside the server**: managed databases, object storage, a log pipeline. A server can disappear at any moment and nothing may be lost with it.
- **Fast image builds**: if a critical CVE patch takes three days to reach production, people will start patching by hand again. Aim for hours, with automatic rebuilds when a base image updates.
- **Safe replacement**: PodDisruptionBudgets, connection draining and surge capacity so replacing every server is a non-event, done routinely (weekly), not once a year.

WORKED EXAMPLE: REBUILDING NODES TO APPLY A KERNEL PATCH

A critical kernel CVE is published. The image pipeline is triggered: it starts from the patched base image, bakes the node agents, runs a vulnerability scan (which now shows the CVE fixed), boots a test cluster from it and runs smoke tests, then publishes **node-image v43**. The Kubernetes node pool is upgraded with a **surge** of one extra node: a new v43 node joins, one old node is **cordoned** (no new pods) and **drained** (pods evicted, respecting each PodDisruptionBudget, so for example only one of three database replicas moves at a time), then deleted. Repeat until every node runs v43. Nobody logs in to a server, and the rollout record shows exactly when each node changed.

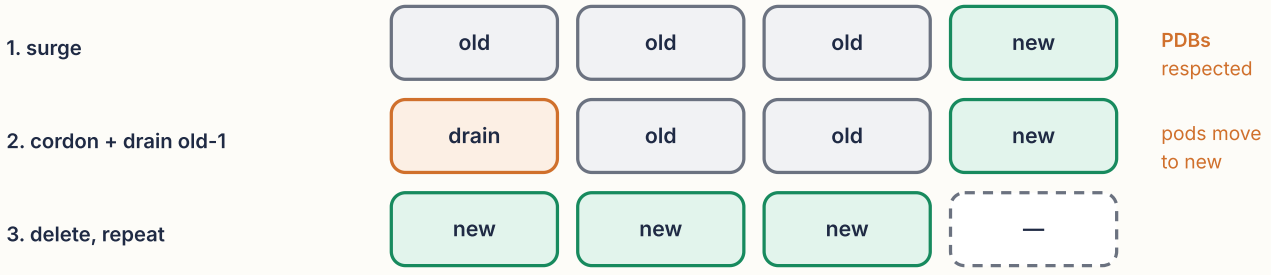


Figure 3. A surge upgrade of a node pool: add a new node, drain an old one into it, delete, repeat. Capacity never drops below normal.

TRADE-OFFS

- Every change, even one setting, costs an image build and a rollout.
- Stateful servers need state moved off them first (lesson 59).
- Large images slow boots and autoscaling; keep images lean and cache layers.
- Needs mature automation: build pipeline, IaC, health checks, observability.

COMMON MISTAKES

- "Immutable" images, then a config tool or SSH session changes servers after boot.
- Secrets or environment-specific config baked into the image.
- Logs written to local disk, lost when the server is replaced.
- Image builds so slow or manual that urgent patches bypass them.

7 Key terms

Snowflake server: unique, hand-modified, unreproducible server

Golden image: versioned, tested image with OS, patches, agents, app

Pets vs cattle: hand-cared-for servers vs identical, replaceable ones

Bake vs fry: configure at build time vs at boot time

Configuration drift: servers diverging from their intended config

Cordon / drain: stop new pods on a node / evict existing ones

8 Interview questions

Q1. Why prefer immutable infrastructure? BASIC

Replacing servers from versioned, tested images instead of patching in place removes drift, makes environments identical and reproducible, simplifies rollback to a known-good image, and lets one image scan cover every server.

Q2. What does it require? BASIC

Automated image builds, state and logs stored outside the servers, and deployments that replace instances safely with health checks.

Q3. What is a golden image? BASIC

A pipeline-built, versioned and tested image containing the OS, patches, agents and usually the app, used to start every server.

Q4. A critical CVE drops on Friday. How does an immutable shop patch 400 servers? SCENARIO

Rebuild the image from the patched base, scan and test it, then roll the fleet by replacement with surge and health checks; nobody SSHes in.

Q5. Developers want SSH to debug production. What do you offer instead? SCENARIO

Good logs, metrics and traces, ephemeral debug containers or a break-glass, audited access path whose changes are never kept; the fix still goes through the pipeline.

Q6. Can a database server be immutable? TRICKY

The server can, if data lives on a separate persistent volume or managed storage: replace the VM or pod, re-attach the volume or replica, and the data outlives the instance.

Q7. Isn't Ansible enough to prevent drift? TRICKY

It reduces drift for what it manages, but anything outside its playbooks still accumulates and servers keep a history; replacement guarantees a known starting state every time.

Next: SD-V-014 · Cloud Cost Optimization backendroadmap.com